

Traditional Python Class vs. Python @dataclass

Here is a comparison highlighting how the **@dataclass** decorator eliminates boilerplate code for data-centric structures.

1. The Traditional Python Class

In a traditional class, you must manually write the constructor (`__init__`), the string representation (`__repr__`), and the structural equality logic (`__eq__`).

```
class TraditionalBook:
    def __init__(self, title: str, author: str, pages: int):
        self.title = title
        self.author = author
        self.pages = pages

    def __repr__(self):
        # Manually formatting the string printout
        return f"TraditionalBook(title={self.title!r}, author={self.author!r}, pages={self.pages})"

    def __eq__(self, other):
        # Manually checking if the other object is the same class and has the same values
        if not isinstance(other, TraditionalBook):
            return NotImplemented
        return (self.title == other.title and
                self.author == other.author and
                self.pages == other.pages)

# Usage
book1 = TraditionalBook("1984", "George Orwell", 328)
book2 = TraditionalBook("1984", "George Orwell", 328)

print(book1)          # Output: TraditionalBook(title='1984', author='George Orwell', pages=328)
print(book1 == book2) # Output: True (Because we wrote __eq__)
```

2. The Python @dataclass Way

With the **@dataclass** decorator, you only list the attributes and their types. Python writes the `__init__`, `__repr__`, and `__eq__` methods for you behind the scenes.

```
from dataclasses import dataclass

@dataclass
class DataclassBook:
    title: str
    author: str
    pages: int

# Usage
book1 = DataclassBook("1984", "George Orwell", 328)
```

```
book2 = DataclassBook("1984", "George Orwell", 328)

print(book1)          # Output: DataclassBook(title='1984', author='George Orwell', pages=328)
print(book1 == book2) # Output: True (Automatically works out of the box)
```

Key Differences in Behavior

- **Line Count:** The traditional class takes 16 lines of code to achieve basic functionality, while the dataclass takes only 6 lines.
- **Default Identity Comparison:** If you deleted the `__eq__` method from the Traditional Class, `book1 == book2` would return False because Python would check object IDs in memory. The Dataclass automatically checks internal values.
- **Maintenance:** If you decide to add a fourth attribute (like `pub_year`) to the traditional class, you have to manually update `__init__`, `__repr__`, and `__eq__`. In the dataclass, you simply add one line: `pub_year: int`.